

Reviewer Pack — Projective Plane Line Count Bounds ABP Width for Monotone Ci...

Entry 96cc863111fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 01:02:16 UTC

Projective Plane Line Count Bounds ABP Width for Monotone Circuits

Entry ID: 96cc863111fe **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 01:02:16 UTC

1. Conjecture

Field A (mathematical branch): Finite Geometry **Field B** (complexity object): Monotone Boolean Circuit Size

Statement:

For every monotone Boolean circuit computing the characteristic function of a projective plane $PG(2, q)$, the minimal width of an ABP computing the same function satisfies $\text{width} \geq q^2 + q + 1$. This bound is tight for $q = 2^k$.

Rationale (proposer's reasoning):

Projective planes exhibit highly symmetric incidence structures that could encode combinatorial constraints mirroring the layered computation of ABPs. The tightness of the bound suggests a deep algebraic relationship between geometric incidence and algebraic computation.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fd49a5c576252a22

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - projective plane ABP width monotone Boolean circuit size - $q^2 + q + 1$
projective plane monotone circuit lower bound - monotone Boolean circuit ABP width $PG(2, q)$
finite geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_projective_plane(q):
    if q < 2:
        raise ValueError("q must be at least 2")

    points = list(range(q**2 + q + 1))
    lines = []

    for i in range(q**2 + q + 1):
        line = [i]
        for j in range(1, q + 1):
            line.append((i * j) % (q**2 + q + 1))
        lines.append(line)

    return points, lines

def adjacency_matrix(points, lines):
    n = len(points)
    adj_matrix = [[0] * n for _ in range(n)]

    for point in points:
        for line in lines:
            if point in line:
                for other_point in line:
                    if other_point != point:
                        adj_matrix[points.index(point)][points.index(other_point)] = 1

    return adj_matrix

def is_connected(matrix):
    n = len(matrix)
    visited = [False] * n
    stack = [0]
```

```

while stack:
    node = stack.pop()
    if not visited[node]:
        visited[node] = True
        for neighbor in range(n):
            if matrix[node][neighbor] == 1 and not visited[neighbor]:
                stack.append(neighbor)

return all(visited)

def min_width_abp(adj_matrix):
    n = len(adj_matrix)
    dp = [[float('inf')] * (n + 1) for _ in range(n)]
    dp[0][1] = 0

    for k in range(2, n + 1):
        for i in range(n):
            for j in range(i + 1, n):
                if adj_matrix[i][j] == 1:
                    dp[j][k] = min(dp[j][k], dp[i][k - 1] + 1)

    return dp[n - 1][n]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

    for n in [5, 10, 15, 20, 30, 40]:
        q = 2 ** (n // (n + 1))
        points, lines = generate_projective_plane(q)

        if not is_connected(adjacency_matrix(points, lines)):
            return {
                "metric_name": "ABP Width",
                "metric_value": None,
                "instances_tested": 0,
                "conjecture_holds": False,
                "counterexample": "mapping_undefined"
            }

        width = min_width_abp(adjacency_matrix(points, lines))
        results.append(width)

    mean = sum(results) / len(results)
    conjecture_holds = all(w >= q**2 + q + 1 for w in results)
    counterexample = "" if conjecture_holds else f"q={q}, expected {q**2+q+1}, got {min(results)}"

    return {
        "metric_name": "ABP Width",
        "metric_value": mean,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(1, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55481e38.py", line 112, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55481e38.py", line 80, in run_trial
    points, lines = generate_projective_plane(q)
                     ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55481e38.py", line 20, in generate_proj
    raise ValueError("q must be at least 2")
ValueError: q must be at least 2

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to invalid q value (must be ≥ 2), preventing evaluation of conjecture |
 next: Run test with q=2 and valid projective plane parameters to verify bounds

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109203
2	propose	ollama_remote	qwen3:8b	0	0	106070
3	preregistration	ollama_remote	qwen3:8b	0	0	24335
4	novelty	ollama_remote	qwen3:8b	0	0	17909
5	novelty	ollama_remote	qwen3:8b	0	0	15443
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16029
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10834
8	judge	ollama_remote	qwen3:8b	0	0	18992

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 318814 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in *research/audit/96cc863111fe.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/96cc863111fe.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/96cc863111fe.tar.gz* (if generated)